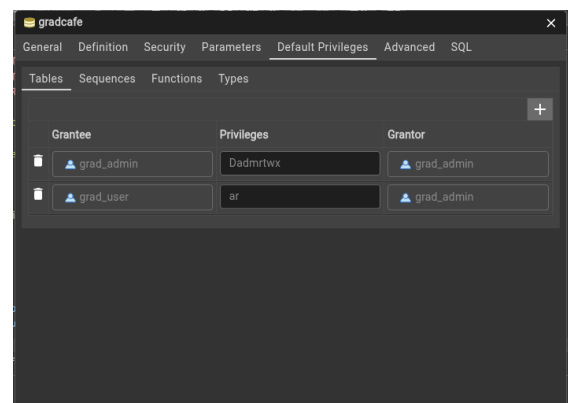
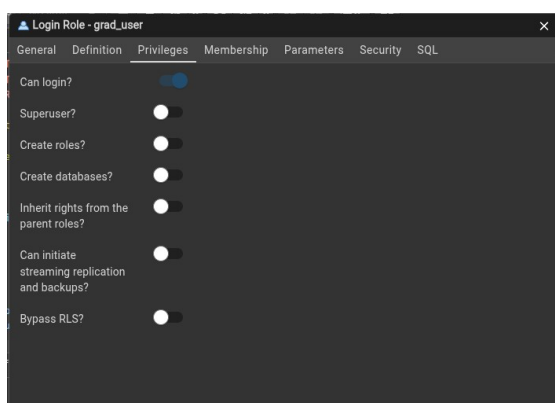


Database Hardening:

I reduced the access rights for grad_user to disallow any database level changes, since this role simply doesn't need to be creating, replicating, or otherwise modifying anything at that level. I then reduced the table level access to only allow reading and writing. The writing function is necessary for the update functionality to work properly, and the read function is necessary to read the data for generating statistics. I did this in pgAdmin, screenshots of these changes are below.

Within the SQL, I added the LIMIT clause to all of the statements that can take it. In this particular case, because the SELECT clauses should only ever return one record, I limited all of them to 1, in case they were somehow manipulated. If I was returning more data, I would use pagination and limit the returns to the quantity needed per page. I was already using parameters in the only INSERT function, since this is a common method of sanitizing unsafe data (the scraped data **should** be safe, since it is coming from gradcafe's database, but any data that we didn't have control over from end-to-end should be assumed risky). There are no other user supplied pieces of data being processed currently.



Why Packaging Matters:

Packaging allows the application to be distributed to other users/developers and allow the code to be run with minimal interference of the individual. The process will adjust any system specific items that could differ between individual systems (such as the location of python being in a different location). This ensures that no matter what system the package is moved to, it will function as expected. This consistency will ultimately save time for everyone using the package and also provides some additional documentation about the package itself.

PyDeps:

As expected, the majority of the work is connected with the flask module, due to all of the code in the background needed to serve HTTP content. Likewise, routes is the primary connector for all of the code that I generated myself. Which makes sense, since it is the gateway to interacting with the rest of the app. I would expect, that as the web app gets larger, more connections would come back to routes (or I would need to break up functionality into different routes based on related functionality). So far, it is relatively simple as far as dependency webs go.

Fresh Install:

Pip

- From the root directory, create a virtual environment: `pip -m venv venv`
- Activate the virtual environment: `source /venv/bin/activate`
- Install dependencies: `pip install -r requirements.txt`
- Edit the environment file with information for this instance, production in `/src/.env` and the test server in `/tests/.env`
- To start the Flask server: `python3 run.py`

- Connect on localhost in your browser: <http://localhost:8080>

uv

- From the root directory, initialize the application: `uv init`
- Install dependencies: `uv add -r requirements.txt`
- Edit the environment file with information for this instance, production in `/src/.env` and the test server in `/tests/.env`
- To start the Flask server: `uv run run.py`
- Connect on localhost in your browser: <http://localhost:8080>

Snyk code test Results:

Scan results below. Medium warning for a potential cross-site scripting (XSS) attack from the JavaScript loading of the analysis data. The warning is technically true and a result of how I chose to update the analysis portion of the website. I suppose I could do some escape magic to make any of the results pulled from the database “HTML safe”, like converting all “<” to “<”. However, I don’t return any information that is directly transferred from GradCafe, only processed and generated data, so an XSS attack isn’t possible in this use case. I could return the information as JSON and build it in the JavaScript, but I would rather build it on the back-end, since this isn’t exactly an API call at this point.

Open Issues

✗ [MEDIUM] DOM-based Cross-site Scripting (XSS)

Finding ID: 7d10d1a8-9484-4815-bf86-651dde8bfcca

Path: `static/js/update.js`, line 57

Info: Unsanitized input from data from a remote resource flows into `innerHTML`, where it is used to dynamically construct the HTML page on client side. This may result in a DOM Based Cross-Site Scripting attack (DOMXSS).

Test Summary

Organization: niteseer

Test type: Static code analysis

Project path: src

Total issues: 1

Ignored issues: 0 [0 HIGH 0 MEDIUM 0 LOW]

Open issues: 1 [0 HIGH 1 MEDIUM 0 LOW]